**Cod_2840-2_ C AVANZADO – ENTORNOS POSIX Y SOCKETS**

Este curso de C Avanzado está orientado a profesionales con conocimientos previos del lenguaje C que necesiten profundizar en aspectos avanzados de programación de sistemas, gestión eficiente de memoria, concurrencia y comunicaciones en entornos Linux/POSIX.

A lo largo de la formación se trabajarán conceptos fundamentales para el desarrollo de aplicaciones robustas y de alto rendimiento, incluyendo programación modular, uso avanzado de punteros, gestión de memoria, sincronización de hilos, programación de sockets y técnicas avanzadas de debugging.

El enfoque es eminentemente práctico y está orientado a entornos reales de desarrollo, permitiendo a las personas participantes adquirir conocimientos aplicables desde el primer momento.

**Objetivos**

- Profundizar en el uso avanzado del lenguaje C.
- Comprender técnicas avanzadas de gestión de memoria.
- Diseñar aplicaciones modulares y reutilizables.
- Trabajar con concurrencia y sincronización mediante POSIX Threads.
- Implementar comunicaciones mediante sockets TCP/UDP.
- Aplicar técnicas avanzadas de debugging y diagnóstico de errores.
- Aplicar buenas prácticas de programación en entornos Linux.

**Bloque 1 — C Avanzado: Punteros, Modularidad y Compilación**

**1.1 Punteros avanzados**

- Aritmética de punteros — desplazamientos, alineamiento, aliasing.
- Punteros a funciones — callbacks, tablas de salto, máquinas de estados.
- Punteros void — uso genérico, conversión segura.
- Restricciones de aliasing y restrict — optimización y comportamiento indefinido.

**1.2 Qualifiers y const-correctness**

- const — puntero a const, const a puntero, const en funciones.
- volatile — acceso a hardware, variables compartidas.
- static y extern — visibilidad, linkage interno/externo.

**1.3 Modularidad y diseño de APIs**

- Separación .h / .c — interfaces limpias, ocultación de implementación.
- Ámbito de funciones y variables — global, local, estático.

**1.4 Compilación avanzada**

- Compilación condicional — #ifdef, #ifndef, #pragma.
- Directivas del preprocesador — macros, includes.

**Bloque 2 — Memoria Dinámica**

**2.1 Gestión de memoria**

- malloc/calloc/realloc/free — patrones correctos y errores típicos.
- Memory leaks y dangling pointers — detección y prevención.
- Fragmentación y estrategias de asignación.
- Valgrind y sanitizers — análisis dinámico.

**Se elimina el anterior apartado 2.2 — Estructuras dinámicas.**

**Bloque 3 — Concurrencia POSIX y Sincronización**

**3.1 Hilos POSIX**

- pthread_create y pthread_join — ciclo de vida del hilo.
- Paso de parámetros — estructuras, memoria compartida.
- Finalización y retorno — pthread_exit, pthread_cancel.

**3.2 Sincronización**

- Mutex — exclusión mutua, deadlocks, buenas prácticas.
- Semáforos POSIX — binarios y contadores.
- Condvars — señalización, espera, productor/consumidor.
- RWLocks y spinlocks — optimización de concurrencia.
- Problemas típicos con real time.

**3.3 Problemas clásicos**

- Race conditions — detección y mitigación.
- Deadlocks y livelocks — estrategias de evitación.
- Productor/Consumidor — implementación completa.

**Bloque 4 — Redes y Programación de Sockets**

**4.1 Fundamentos de sockets**

- Sockets TCP y UDP — flujo vs datagramas.
- bind, listen, accept — servidor básico.
- connect, send, recv — cliente básico.

**4.2 Multiplexación de E/S**

- select — monitorización de múltiples sockets.
- poll — alternativa escalable.
- epoll — alto rendimiento en Linux.

**4.3 Servidores concurrentes**

- Modelo por hilos — un hilo por cliente.
- Modelo por procesos — fork().
- Modelo reactivo — select/epoll.

**4.4 Protocolos y serialización**

- Protocolos binarios — endianess, alineamiento.
- Serialización — estructuras empaquetadas.

**Bloque 5 — Debugging avanzado con gdb y herramientas del sistema**

**5.1 Debugging avanzado**

- Uso avanzado de gdb: breakpoints condicionales, watchpoints y TUI.
- Análisis de fallos con core dumps.
- Trazado con strace/ltrace.
- Análisis de memoria con Valgrind y sanitizers.
- Diagnóstico de concurrencia y herramientas del sistema para detectar cuellos de botella, corrupción de memoria y problemas de sincronización.

**Duración total: 25 horas, distribuidas en 5 jornadas de 5 horas.**